Practical : 3

AIM:

To implement Word Count using Hadoop HDFS and MapReduce for large text datasets.

PROBLEM DEFINITION:

Scenario: A meteorological research organization maintains historical weather records collected from various weather stations worldwide. The organization uses Hadoop to analyze large climate datasets and study long-term temperature trends. As a Data Analytics Engineer, you are required to process the National Climatic Data Center (NCDC) dataset and identify the maximum temperature recorded for each year using MapReduce

GitHub Link:
https://github.com/24dit061-Satyarajsinh/24DIT061_CSUE301_BDA

OUTPUT :

Task 1: Configure Hadoop + Upload Dataset

```
!jps
```

```
[0.001s][warning][os,contai
[0.001s][warning][os,contai
4448 ResourceManager
11713 MRAppMaster
11330 NodeManager
12150 Jps
2759 NameNode
2824 DataNode
2895 SecondaryNameNode
```

```
!wc -l sample.txt
```

```
7 sample.txt
```

Task 2: Mappar

```
%%writefile mapper.py

import sys
import re

for line in sys.stdin:
    line = line.strip().lower()

    words = re.findall(r'\b\w+\b', line)

    for word in words:
        print(f"{word}\t1")
```

```
Writing mapper.py
```

```
!echo "Hadoop is powerful Hadoop is fast" | python3 mapper.py
```

```
hadoop  1
is      1
powerful        1
hadoop  1
is      1
fast    1
```

Task 3: Reducer

```
%%writefile reducer.py

import sys

current_word = None
current_count = 0

for line in sys.stdin:

    line = line.strip()

    if not line:
        continue

    word, count = line.split('\t', 1)
    count = int(count)

    if current_word == word:
        current_count += count

    else:

        if current_word is not None:
            print(f"{current_word}\t{current_count}")

        current_word = word
        current_count = count

if current_word is not None:
    print(f"{current_word}\t{current_count}")
```

```
Writing reducer.py
```

```
!echo "Hadoop is powerful Hadoop is fast Hadoop" | python mapper.py | sort | python reducer.py
```

```
fast    1
hadoop  3
is      2
powerful        1
```

Task 4: Execute MapReduce + Top 10

```
%%bash

export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64
export HADOOP_HOME=/content/hadoop-3.4.1
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop

$HADOOP_HOME/bin/hdfs dfs -ls /output
```

```
[0.001s][warning][os,container] Cgroup memory controller path at '/sys/fs/cgrou
[0.001s][warning][os,container] Cgroup cpu controller path at '/sys/fs/cgroup'
Found 2 items
-rw-r--r--   1 root supergroup          0 2026-08-23 04:41 /output/_SUCCESS
-rw-r--r--   1 root supergroup         55 2026-08-23 04:41 /output/part-00000
```

```
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64
export HADOOP_HOME=/content/hadoop-3.4.1
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop

$HADOOP_HOME/bin/hdfs dfs -cat /output/part-00000
```

```
[0.001s][warning][os,container] Cgroup memory controller pa
[0.001s][warning][os,container] Cgroup cpu controller path
apache  1
big     1
colab   1
data    1
hadoop  4
hello   3
world   1
```

```
%%bash

export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64
export HADOOP_HOME=/content/hadoop-3.4.1
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop

$HADOOP_HOME/bin/hdfs dfs -cat /output/part-* | \
sort -k2,2nr | \
head -10
```

```
hadoop  4
hello   3
apache  1
big     1
colab   1
data    1
world   1
[0.002s][warning][os,container] Cgroup cpu controller path at '/sys/fs/cgr
[0.002s][warning][os,container] Cgroup memory controller path at '/sys/fs/
```

Task 5: Performance Evaluation

```
import pandas as pd

data = {
    "Dataset": ["Small", "Medium", "Large"],
    "Processing Time (seconds)": [41, 44, 40]
}

df = pd.DataFrame(data)

df
```

	Dataset	Processing Time (seconds)
0	Small	41
1	Medium	44
2	Large	40

Next steps: Generate code with df | New interactive sheet

KEY QUESTIONS :

Q1. What is the role of Mapper and Reducer in the Word Count application?

The Mapper reads the input text and generates key-value pairs in the form (word, 1). The Reducer groups the same words together and adds their values to calculate the total count. Together, Mapper and Reducer help process the text efficiently using Hadoop MapReduce.

Q2. How are intermediate key-value pairs generated and processed?

The Mapper splits each line into individual words and generates pairs such as (Hadoop, 1). Hadoop groups all the same words together before sending them to the Reducer. The Reducer then adds all the values for each word and produces the final count.

Q3. How does HDFS store and distribute the input dataset?

HDFS divides the input file into blocks and stores them across DataNodes. The blocks can be replicated to provide reliability and fault tolerance. This distributed storage allows Hadoop to process large datasets efficiently.

Q4. What advantages does MapReduce offer for processing large datasets?

MapReduce processes large datasets by dividing the work into smaller tasks that can run in parallel. It can handle very large amounts of data and distribute the processing across multiple nodes. It also provides fault tolerance by allowing failed tasks to be executed again**.**

Q5. How does the execution time vary with dataset size?

As the size of the dataset increases, the execution time generally increases. MapReduce can reduce the impact of large data sizes by processing different parts in parallel. For very small datasets, Hadoop's startup overhead may make execution time relatively higher.

CONCLUSION :

This practical successfully demonstrated the use of Hadoop HDFS and MapReduce for Word Count analysis. The Mapper generated intermediate key-value pairs, while the Reducer calculated the frequency of each word. HDFS provided distributed storage, and MapReduce made it possible to process large datasets efficiently with execution time increasing as the dataset size increased.